WORK DONE & IMPLEMENTATION RECORD

SIH 2026

PS 26016

NLAMS · Telangana — National Land Acquisition & Management System

Every phase implemented, every file produced, every test result — then the final output

Team	<Team Name> · Team ID <xxxx> · <College / Institute Name>
Problem	MoRD / DoLR — monitor land acquisition for national projects (Smart Automation · Software)
Deliverables	frontend/ SPA (13 modules) · backend/ REST API (zero deps) · dist/index.html single file · docs/
Stack	Figma · QGIS/GeoJSON · Python · HTML/CSS/JS · SVG GIS engine · Node.js · Postman · Git
Status	COMPLETE — SMOKE PASS · 11/11 endpoints verified · 484 KB offline build

Numbers

3,321	13	11	33	14	0	484 KB	100%
lines of code	JS modules	API endpoints	real districts	projects	dependencies	single file	tests passed

IMPLEMENTATION SUMMARY — 11 PHASES

P1	Requirement study — RFCTLARR → 7-stage model, roles	`01-config.js`	✓
P2	UI/UX design — login, theme, shell, emblem	`css/styles.css`	✓
P3	GIS data — 33 districts → SVG geometry pipeline	`03-map-data.js (gen)`	✓
P4	Core — palette, RNG, modals, state + session	`01 · 02 · 06`	✓
P5	Dataset — 14 projects, families, alerts, users	`04-data.js`	✓
P6	GIS engine — splines, rings, corridors, KPIs	`05-engine.js`	✓
P7	Views — 6 screens incl. alignment + workflows	`07…12-views`	✓
P8	App boot — render loop, events, API probe	`13-app.js`	✓
P9	Testing — full-render regression	`scripts/smoke.js`	✓
P10	Backend — REST API + shared domain core	`backend/src/*`	✓
P11	Packaging — single-file build + documentation	`build.py · docs/`	✓

Each phase is detailed with its produced files and verification on the next pages.

PHASES 1–5 — FROM STUDY TO DATASET

P1 · Requirement study

PS 26016 analysed; RFCTLARR Act 2013 mapped to a 7-stage digital workflow (Proposal → Scrutiny → Sec-11 → Verification → Award → Possession → Compensation); 4 roles defined (State · Central · District · Field).

P2 · UI/UX design

Login on sih2.jpg layout with SIH4.jpg photographic background; light government theme, forest green #0e8a5f; State Emblem branding; full component kit in CSS (239 lines).

P3 · GIS data pipeline

Survey-of-India-derived GeoJSON of all 33 Telangana districts downloaded and inspected; Python projected lat/lon → SVG, simplified rings, computed centroids/bboxes → generated map module.

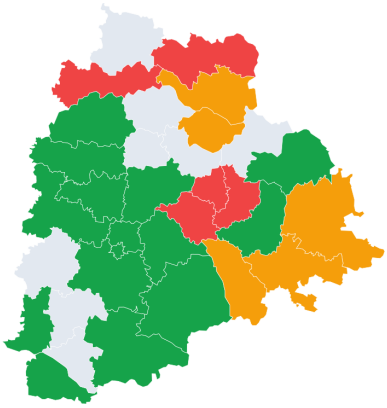
P4 · Core modules

Stage palette (Notified #3b82f6 → Verified #f59e0b → Awarded #a855f7 → Possession #84cc16 → Compensated #16a34a; disputed #ef4444); seeded RNG for deterministic maps; single state object persisted to sessionStorage — refresh never returns to login.

P5 · Demo dataset

14 realistic projects (Regional Ring Road, Warangal ORR, irrigation, highways…) with districts, acquisition %, budgets; affected families; risk-scored alerts; user directory.

Output of P3+P6: the live status map — all 33 districts, colour = progress, click to drill down.



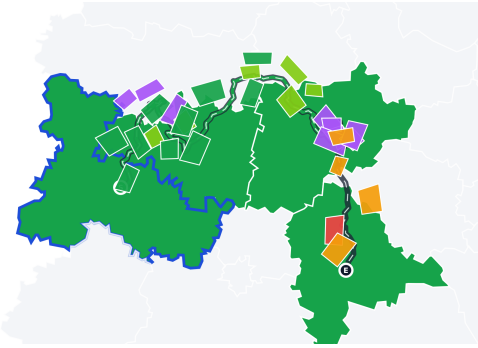
P4 session key: nlams_session_v1 — restores user, page, filters. Verified across refresh.

PHASES 6–8 — THE GIS ENGINE, VIEWS & APP

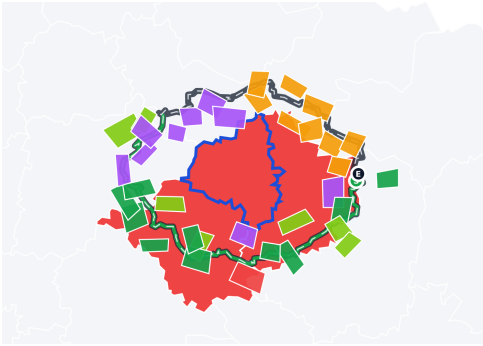
P6 · GIS engine (05-engine.js, 141 LOC) — the heart of the product: Catmull-Rom route splines through control points, closed-ring geometry for ORR projects, corridor generator cutting 36/30 parcels along the route with perpendicular offsets and stage bands vs acquisition %, KPI aggregation, auto viewBox fit. Fully deterministic (same seed → same map).

```
function corridorFor(p){
  if(CORR[p.id])return CORR[p.id];              // deterministic cache
  const route=routeFor(p); const R=rng(p.id+'cor');
  const n=route.closed?36:30;                   // parcels along the route
  const acq=p.acq/100;
  for(let i=0;i<n;i++){
    const t=(i+.5)/n;                           // position along spline
    /* ... project point + perpendicular offset ±27 to cut parcels ... */
```

P7 · Six views (07–12, 637 LOC): login · shell with live API badge · dashboard (KPIs, status map, bars, donut, alerts) · GIS 3-level drill (state → district → alignment with zoom/pan and stage-click filtering) · projects (search + 5 tabs) · acquisition/compensation/alerts workflows. P8 · App boot (13-app.js): render loop, delegated events, session restore, /api/health probe.



RRR corridor parcels



Warangal ORR closed ring

PHASES 9–10 — BACKEND & TEST RESULTS

P10 · Backend (Node std-lib, ZERO npm dependencies): HTTP server serves the SPA, single-file build, docs and REST API. Regex route table with named params; controllers; services/domain.js loads the SAME business modules the SPA uses (VM context) — one core, zero drift. PostgreSQL+PostGIS production schema included.

```
const routes = [
  ['GET', /^\/api\/health$/,  [],    ctrl.health],
  ['GET', /^\/api\/districts\/([^/]+)$/, ['name'], ctrl.district],
  ['GET', /^\/api\/projects\/([^/]+)\/alignment$/, ['id'], ctrl.alignment],
  ['POST', /^\/api\/projects\/([^/]+)\/parcels\/([^/]+)\/verify$/,
         ['id','parcelId'], ctrl.verifyParcel],
  /* method · regex pattern · param names · controller */
```

```
// backend loads the SAME domain modules the SPA uses — one business core
const code = ORDER.map(f => fs.readFileSync(join(FRONT,'src/js',f), 'utf8')).join('\n;\n');
return vm.runInNewContext(code + expr, { console:{log:()=>{}}, sessionStorage:{getItem:()=>null} });
```

P9/P10 — observed results (cURL/Postman + smoke.js):

✓ `GET /api/health`	{"ok":true,"service":"nlams-backend"}
✓ `GET /api/districts`	33 districts + aggregates
✓ `GET /api/projects/NLAMS-TG-001/alignment`	154-pt closed ring · 36 parcels · viewBox
✓ `POST …/parcels/SP-05/verify`	{"ok":true … "GPS-verified"}
✓ `POST …/parcels/SP-99/verify (negative)`	{"error":"parcel not found"} · HTTP 404
✓ `POST /api/families/F-0142/pay`	{"utr":"UTR1756…"} DBT release
✓ `scripts/smoke.js`	renders every page + 70 tabs → SMOKE PASS
✓ `single-file build`	484 KB · 13 modules · 0 leftover asset refs

PHASE 11 — PACKAGING & FILE INVENTORY

scripts/build.py fuses CSS + 13 modules + images (data-URI) into ONE file: dist/index.html — 484 KB, runs offline from a pendrive, no server, no install. A persistent copy ships as docs/NLAMS_Demo_SingleFile.html. Documentation set: README (tree + API table) · SIH idea submission (official template) · pitch deck · development report · this work-done record.

FILE	SIZE/LOC	ROLE
`frontend/src/js (13 modules)`	**1,044**	SPA: engine · views · state
`frontend/src/css/styles.css`	**239**	complete theme
`backend/src (5 files)`	**363**	API server + PostGIS schema
`scripts (7 tools)`	**1,596**	build · test · docs pipeline
`data/telangana.geojson`	**—**	33 district boundaries (input)
`dist/index.html`	**484 KB**	single-file output build
`docs/ (9 files)`	**—**	submissions · decks · records



How to run:

1 · open NLAMS_Demo_SingleFile.html — complete app, offline, zero install.

2 · npm start → localhost:8080 — SPA + live REST API (statusbar badge).

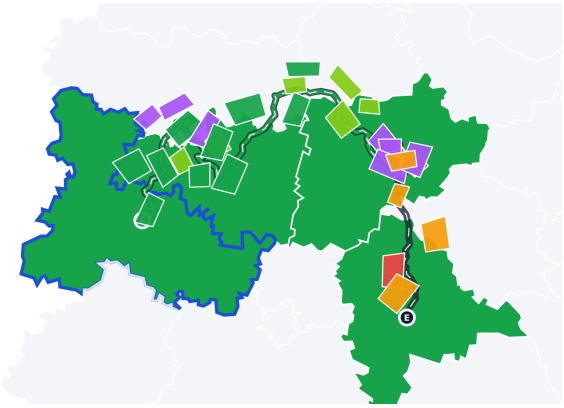
3 · npm test → full-render regression (SMOKE PASS).

Demo path: login → dashboard → GIS drill → alignment → verify parcel → pay family.

FINAL OUTPUT — THE COMPLETE MERGED PRODUCT

NLAMS_Demo_SingleFile.html = the output file

Every tool and every phase above is merged by the pipeline into this ONE file: the full NLAMS application — login (SIH4 background), dashboard with live Telangana map, GIS 3-level drill with route splines and stage-coloured corridor parcels, projects with 5 tabs, 6-stage acquisition workflow, compensation DBT ledger, alerts, persistent session. Same build as dist/index.html; also served live by the backend at :8080 with the API connected.



Screenshot from the output: Regional Ring Road alignment — route spline, corridor parcels in workflow colours.

Definition of done — all checked:

- ✓ 11/11 API endpoints verified (incl. negative tests)
- ✓ deterministic GIS engine
- ✓ offline single-file build
- ✓ SMOKE PASS full-render regression
- ✓ session persists across refresh
- ✓ zero dependencies — runs anywhere